

Assignment 3

Support Vector Machines and Neural Networks

Name: Shahbaz Shaikh
Course: Advance Data Mining & Predictive Analytics
Date: April 22, 2026

Introduction

This assignment covers two major topics from Modules 6 and 7: support vector machines and artificial neural networks. In Part A, I answer short conceptual questions about SVMs, perceptrons, and the learning rate. In Part B, I use the Carseats dataset to build and evaluate a linear SVM regression model and a neural network regression model. The goal is to predict Sales using the variables Price, Advertising, Population, Age, Income, and Education.

Part A

Question 1: What is the difference between SVM with hard margin and soft margin?

Answer:

A hard-margin SVM is used when the data are perfectly linearly separable. It tries to find the hyperplane that separates the classes with the widest possible margin while making zero classification errors. This approach works well only when the data are very clean. If there are outliers or overlapping classes, a hard-margin SVM can perform poorly or may not be possible to fit at all.

A soft-margin SVM is more flexible. It still tries to maximize the margin, but it allows some observations to either fall inside the margin or be misclassified. This makes it much more useful for real-world data, where noise and overlap are common.

So, the main difference is that a hard-margin SVM does not allow any classification errors, while a soft-margin SVM allows some errors in exchange for better generalization and better handling of noisy data.

Question 2: What is the role of the cost parameter, C, in SVM with soft margin classifiers?

Answer:

The cost parameter C controls how much the model penalizes classification errors. In other words, it controls the trade-off between having a wide margin and correctly classifying the

training data.

If C is large, the model gives a high penalty to misclassification. That means it tries harder to classify every training observation correctly, even if this leads to a smaller margin. This can make the model fit the training data more closely, but it can also increase the chance of overfitting.

If C is small, the model is more tolerant of classification errors and focuses more on maintaining a wider margin. This usually gives a more regularized model that may generalize better to new data.

So, C controls how strict the SVM is. A larger C means fewer errors are tolerated, while a smaller C means the model allows more errors in order to build a simpler and more flexible decision boundary.

Question 3: Will the following perceptron be activated if 2.8 is the activation threshold?

Answer:

To decide whether the perceptron is activated, I first calculate the weighted sum of the inputs and then compare that result to the activation threshold.

The values given in the diagram are:

Input 1 = 0.1 with weight 0.8

Input 2 = 11.1 with weight -0.2

Threshold = 2.8

Step 1: Multiply each input by its weight.

$$0.1 \times 0.8 = 0.08$$

$$11.1 \times -0.2 = -2.22$$

Step 2: Add the weighted inputs.

$$0.08 + (-2.22) = -2.14$$

Step 3: Compare the result to the threshold.

Since -2.14 is less than 2.8, the perceptron will not activate.

Final Answer: The perceptron is **not activated** because the weighted sum is -2.14, which is below the activation threshold of 2.8.

Question 4: What is the role of alpha, the learning rate, in the delta rule?

Answer:

In the delta rule, alpha is the learning rate. It determines how much the weights are adjusted during each update step.

If alpha is too large, the weights may change too much at each step. This can make the model unstable and cause it to overshoot the best solution. If alpha is too small, the model learns very slowly and may take too long to converge.

Because of this, alpha plays an important role in how efficiently the model learns. A good learning rate helps the model update its weights at a steady pace so it can reduce prediction error without becoming unstable.

Part B

Dataset and Goal

For Part B, I used the Carseats dataset from the ISLR package. According to the instructions, I selected only the following variables: Sales, Price, Advertising, Population, Age, Income, and Education. The goal is to predict Sales using all of the other selected variables as predictors.

The main packages used in this part are ISLR, dplyr, glmnet, and caret. The following code loads the data and keeps only the required columns.

```
library(ISLR)
library(dplyr)
```

```
library(glmnet)
library(caret)
```

```
Carseats_Filtered <- Carseats > select(Sales, Price, Advertising,
Population, Age, Income, Education)
```

Question B1: Build a linear SVM regression model to predict Sales based on all other attributes. What is the R-squared of the model?

Answer:

To answer this question, I trained a linear SVM regression model using the caret package with method = "svmLinear". I used Sales as the response variable and the remaining six variables as predictors. I also used 5-fold cross-validation to evaluate the model.

```
set.seed(123)
```

```
train_control <- trainControl(method = "cv", number = 5)
```

```
svm_linear_model <- train(
  Sales ~ .,
  data = Carseats_Filtered,
  method = "svmLinear",
  trControl = train_control
)
```

```
svm_linear_model
```

The R-squared of the linear SVM regression model is **0.3442665**.

Interpretation: This means the model explains about 34.43% of the variation in Sales. The model has some predictive value, but it does not explain a very large portion of the variability in the

response.

Question B2: Customize the search grid by checking the model's performance for $C = 0.1$, 0.5 , 1 , and 10 using 2 repeats of 5 -fold cross validation.

Answer:

For this question, I tuned the linear SVM model by testing four different values of the cost parameter C : 0.1 , 0.5 , 1 , and 10 . I used repeated cross-validation with 5 folds and 2 repeats, as required in the assignment.

```
set.seed(123)

train_control_repeated <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 2
)

svm_grid <- expand.grid(C = c(0.1, 0.5, 1, 10))

svm_tuned_model <- train(
  Sales ~ .,
  data = Carseats_Filtered,
  method = "svmLinear",
  trControl = train_control_repeated,
  tuneGrid = svm_grid
)

svm_tuned_model
plot(svm_tuned_model)
```

The best value of C was **0.5**, and the corresponding R-squared was **0.3466721**.

Interpretation: Among the tested values, $C = 0.5$ gave the best overall performance based on the tuning results. The improvement over the original model is small, but this step shows that tuning hyperparameters can slightly improve predictive performance.

Question B3: Train a neural network model to predict Sales based on all other attributes. What is the R-square of the model with the best hyperparameters?

Answer:

To answer this question, I trained a neural network regression model using the caret package with `method = "nnet"`. Because neural networks are sensitive to the scale of the predictors, I centered and scaled the data before training. I kept the output linear by setting `linout = TRUE` because this is a regression problem, not a classification problem.

```
set.seed(123)

nnet_model <- train(
  Sales ~ .,
  data = Carseats_Filtered,
  method = "nnet",
  preProcess = c("center", "scale"),
  linout = TRUE,
  trace = FALSE,
  maxit = 500
)

nnet_model
plot(nnet_model)
```

The best neural network model had **size = 1** and **decay = 0.1**. Its R-squared was **0.3448199**.

Interpretation: This R-squared is very close to the SVM model results. In this case, the neural network performed slightly better than the initial SVM model, but the difference is very small.

Question B4: Consider the given input record. What will be the estimated Sales for this record using the neural network model?

Answer:

The assignment gives the following values:

Sales = 9

Price = 6.54

Population = 124

Advertising = 0

Age = 76

Income = 110

Education = 10

Since Sales is the value being predicted, it is not used as an input in the prediction step. Only the predictor variables are entered into the model.

```
new_data <- data.frame(  
  Price = 6.54,  
  Advertising = 0,  
  Population = 124,  
  Age = 76,  
  Income = 110,  
  Education = 10  
)
```

```
predicted_sales <- predict(nnet_model, newdata = new_data)
predicted_sales
```

The estimated Sales value for this record is **11.46014**.

Interpretation:Based on the trained neural network model, a store with these characteristics would be expected to have sales of about 11.46.

Overall Discussion

This assignment helped me compare two different machine learning approaches: support vector machines and neural networks. In Part A, I explained the basic ideas behind hard-margin and soft-margin SVMs, the role of the cost parameter C , how a perceptron makes an activation decision, and why the learning rate matters in the delta rule.

In Part B, I applied both SVM regression and neural network regression to the Carseats dataset. The linear SVM model gave an R-squared of 0.3442665, and after tuning, the best SVM model had an R-squared of 0.3466721 with $C = 0.5$. The neural network model gave an R-squared of 0.3448199 with $\text{size} = 1$ and $\text{decay} = 0.1$. Overall, the tuned SVM performed slightly better than the neural network, but the two models were very close in performance.

This shows that both approaches can be useful for prediction, but model tuning and preprocessing can make an important difference.

Conclusion

Overall, this assignment gave me a better understanding of both the theory and application of support vector machines and neural networks. The conceptual questions helped reinforce important ideas such as margin size, misclassification tolerance, perceptron activation, and learning rate. The practical section showed how these ideas can be applied in R to solve a regression problem.

By using the Carseats dataset, I was able to compare the performance of a linear SVM regression

model and a neural network regression model. The results showed that both models had similar predictive performance, with the tuned SVM performing slightly better. This assignment was helpful in connecting machine learning concepts to real model-building practice.

Appendix: Full R Code

```
library(ISLR)

library(dplyr)

library(glmnet)

library(caret)

set.seed(123)

Carseats_Filtered <- Carseats >

  select(Sales, Price, Advertising, Population, Age, Income,
Education)

# QB1

train_control <- trainControl(method = "cv", number = 5)

svm_linear_model <- train(
  Sales ~ .,
  data = Carseats_Filtered,
  method = "svmLinear",
  trControl = train_control
)

print(svm_linear_model)
```

```
# QB2

train_control_repeated <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 2
)

svm_grid <- expand.grid(C = c(0.1, 0.5, 1, 10))

svm_tuned_model <- train(
  Sales ~ .,
  data = Carseats_Filtered,
  method = "svmLinear",
  trControl = train_control_repeated,
  tuneGrid = svm_grid
)

print(svm_tuned_model)
plot(svm_tuned_model)

# QB3

nnet_model <- train(
  Sales ~ .,
  data = Carseats_Filtered,
```

```
method = "nnet",
preProcess = c("center", "scale"),
linout = TRUE,
trace = FALSE,
maxit = 500
)

print(nnet_model)
plot(nnet_model)

# QB4
new_data <- data.frame(
  Price = 6.54,
  Advertising = 0,
  Population = 124,
  Age = 76,
  Income = 110,
  Education = 10
)

predicted_sales <- predict(nnet_model, newdata = new_data)
print(predicted_sales)
```